

MadStree: tree-level dS integrals, dlog-form differential equations and package usage

MadStree package documentation

August 3, 2026

Abstract

We present the formulas underlying the Wolfram Language package **MadStree** (v0.5), which computes master integrals, iterative reduction data and dlog-form differential equations for arbitrary massive/massless mixed de Sitter (dS) tree integrals, and evaluate them numerically through the Python/FLINT backend FlintNDE. The derivation follows the notation of Chen and Feng [1], Chen et al. [2]: vertex integrals are organized into Kronecker sums of two-dimensional Hankel systems, the differential equations are written in the form $dI = d\Omega \cdot I$, and the boundary conditions are set at the large-damping-energy limit by nested blow-up. We first derive the formulas, then describe how to use the package.

Contents

I	Basic formulas	2
1	Conventions and local function systems	2
1.1	The two-dimensional h -system	2
1.2	Vertex integrals and their Kronecker-sum IBP	3
2	The full-graph building blocks and package conventions	3
2.1	From vertex factors to the full graph	3
2.2	Massless full edges: from four states to a shared two-state	4
3	Iterative reduction and the dlog-form differential equation	5
3.1	Iterative reduction	5
3.2	The dlog-form differential equation	5
3.3	Package conventions	6
4	Boundary conditions by nested blow-up	7
II	Using the package	7
5	Overview, requirements and loading	7
5.1	Requirements and loading	8
5.2	System requirements	8
5.3	Loading the package	8
6	Input and core computation	8
6.1	Tree topology: <code>MSInitTree</code>	8
6.2	Time-only graphs: <code>MSInitTimeGraph</code>	9
6.3	Single-vertex families: <code>MSInitVertexFamily</code>	9
6.4	Core computation	10

7	Boundary data and numerical evaluation	10
7.1	Generating the boundary	10
7.2	Numerical transport	11
7.3	Reading the result	11
8	Outputs, examples, validation and common pitfalls	12
8.1	Output organization	12
8.2	Examples	12
8.3	Validation status and limitations	12
8.4	Common pitfalls	13

Part I

Basic formulas

1 Conventions and local function systems

We follow the conventions of Chen and Feng [1]: the conformal time is $\tau < 0$, $z = -k\tau$ for a line of momentum k , and the two independent solutions of the Bessel equation are

$$h(\nu, 0; z) = z^{-\nu} H_\nu(z), \quad h(\nu, 1; z) = -\partial_z h(\nu, 0; z), \quad (1)$$

where H_ν is the Hankel function. MadStree fixes $\nu = |\nu|$, uses the single function name h , and instead of (1) defines

$$h(\nu, 0; z) = z^{\pm\nu} H_\nu(z), \quad (2)$$

where the plus sign is the default (`NuConvention->"Positive"`) and the minus sign ("`Negative`") is the paper convention. The two bases differ by the factor $z^{2\nu}$. Because the Bessel equation contains only ν^2 , all formulas of one convention are obtained from the other by the uniform replacement

$$\nu|_{2401} \longrightarrow -\nu, \quad (3)$$

applied to every occurrence of ν (equations of motion, M_1 , contact powers, recurrences and the differential equations). Equivalently, the positive-prefactor basis is the paper's function with negative order, since $H_{-\nu} \propto H_\nu$ up to a kinematics-independent phase.

For a vertex v we write $k_{0,v}$ and $\nu_{0,v}$ for its damping energy and time power, and for a line e we write k_e (or simply k_i) and ν_i for its momentum and Bessel order. The propagator of a massive or massless full line is denoted by $G^{\sigma_e \sigma'_e}$ with $\sigma_e = \pm$; we use $\sigma_e = +1, -1$ for G^{++}, G^{--} . A theta-free exponential factor is written as $e^{i\chi_b q_b \tau}$, with $\chi_b = \pm 1$; its only role is to contribute to the signed phase energy

$$p_v = \sum_{b: v(b)=v} \chi_b q_b, \quad (4)$$

so that the vertex time exponent is $e^{ip_v \tau}$. Pure vertex energies, massless cross/external branches and auxiliary regulators are all such factors. The package's public time power is $(-\tau_v)^{A_v}$, so that $\partial_{\tau_v} (-\tau_v)^{A_v} = -A_v (-\tau_v)^{A_v-1}$.

We now turn to the two-dimensional h -system, the second local building block, and to the Kronecker-sum form of the single-vertex IBP.

1.1 The two-dimensional h -system

With $z = -k\tau$, the vector $\mathbf{h} = (h(\nu, 0; z), h(\nu, 1; z))^T$ obeys the first-order system

$$\partial_\tau \mathbf{h} = \left[-ik\sigma_2 - \frac{2\nu+1}{\tau} \frac{\mathbb{I} - \sigma_3}{2} \right] \mathbf{h}, \quad (5)$$

where σ_2, σ_3 are Pauli matrices. In the dimensionless variable z ,

$$\partial_z^2 h(\nu, 0; z) + \frac{2\nu + 1}{z} \partial_z h(\nu, 0; z) + h(\nu, 0; z) = 0, \quad (6)$$

with constant term 1; the physical k^2 is recovered only through $\partial_\tau = -k\partial_z$. For the default positive prefactor and $\nu = 1/2$,

$$z^{1/2} H_{1/2}^{(1,2)}(z) \propto e^{\pm iz}, \quad (2\nu + 1)|_{2401} \xrightarrow{\nu \rightarrow -\nu} 1 - 2\nu = 0, \quad (7)$$

so the first-derivative state is a branch eigenstate, $h^{(1,2)}(1/2, 1) = \pm i h^{(1,2)}(1/2, 0)$.

1.2 Vertex integrals and their Kronecker-sum IBP

For a single vertex with p massive/full legs, the integrals are

$$V_v(\nu_{0,v}, \mathbf{a}) = \int d\tau_v \tau_v^{\nu_{0,v}} e^{ip_v \tau_v} \prod_{i=1}^p h(\nu_i, a_i; -k_i \tau_v), \quad a_i \in \{0, 1\}, \quad (8)$$

and we collect them into the vector $\mathbf{f}_v(a_0) = (V_v(\nu_{0,v} + a_0, \mathbf{a}))_{\mathbf{a}}$. In the paper Chen and Feng [1], the IBP is

$$[M_1 + \nu_{0,v} \mathbb{I}] \mathbf{f}_v(-1) + [M_0 + ip_v \mathbb{I}] \mathbf{f}_v(0) = 0, \quad (9)$$

with the local matrices

$$M_1 = - \sum_{i=1}^p (2\nu_i + 1) \frac{\mathbb{I} - \sigma_3^{(i)}}{2}, \quad M_0 = \sum_{i=1}^p (-ik_i \sigma_2^{(i)}), \quad (10)$$

where $A^{(i)}$ acts on the two-dimensional factor of leg i and trivially elsewhere. Both are diagonalized by the constant matrix

$$T = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & -i \\ -i & 1 \end{pmatrix}, \quad T \sigma_2 T^{-1} = \sigma_3, \quad (11)$$

in the sense that $T_p M_0 T_p^{-1}$ with $T_p = T^{\otimes p}$ is diagonal and M_1 is already diagonal in the binary basis. One lowering and one raising step are therefore

$$A_-(\nu_0) = -M_1(\nu_0)^{-1} M_0, \quad A_+(\nu_0) = -T_p^{-1} \widetilde{M}_0^{-1} T_p M_1(\nu_0 + 1), \quad (12)$$

with $\widetilde{M}_0 = T_p M_0 T_p^{-1}$; only diagonal matrices are inverted.

2 The full-graph building blocks and package conventions

2.1 From vertex factors to the full graph

Each active massive endpoint is a two-dimensional h -slot, each uncontracted massless full edge a shared two-dimensional slot (Sec. 2.2), and each theta-free exponential a one-dimensional factor contributing only to (4). The full-graph space of a sector s is the tensor product of these factors, with dimension $2^{|H_s| + |E_s|}$, where H_s are the active massive endpoint slots and E_s the uncontracted massless edges. We call the bookkeeping of slots and their vertex incidence the *building-block registry*.

For a vertex v the global regular matrices are the sums of the local atoms (10) embedded into the common space:

$$M_{1,v} = \nu_{0,v} \mathbb{I} - \sum_{i \in H_v} (2\nu_i + 1) \frac{\mathbb{I} - \sigma_3^{(i)}}{2}, \quad (13)$$

$$M_{0,v} = ip_v \mathbb{I} + \sum_{i \in H_v} (-ik_i \sigma_2^{(i)}) + \sum_{e \in E_v} (-i\epsilon_{ve} k_e \sigma_2^{(e)}), \quad (14)$$

where $\epsilon_{ve} = +1, -1$ if v is the first, second endpoint of the ordered edge e , and $\sigma_2^{(e)}$ acts on the shared slot of e . The massless edge does not enter M_1 , because the default substitution (3) gives $2\nu + 1 \rightarrow 1 - 2\nu = 0$ for $\nu = 1/2$. All $M_{0;v}$ commute pairwise, so a single constant $U_s = \bigotimes T$ diagonalizes them, giving the energy letters

$$\kappa_{v;\mathbf{a},\mathbf{r}} = p_v + \sum_{i \in H_v} (2a_i - 1)k_i + \sum_{e \in E_v} \epsilon_{ve}(2r_e - 1)k_e, \quad (15)$$

with $a_i, r_e \in \{0, 1\}$ the binary state of the massive and shared slots. The same shared bit r_e enters the letters of both endpoints with opposite signs. The common diagonalization is legitimate only if, on every slot, the local operators of all vertices contain a single Pauli direction (σ_2 for massive and paper- h shared slots, σ_1 for the package quotient of Sec. 2.2); otherwise the fast diagonal inversion below is disabled.

The remaining building block is the theta-carrying massless full edge. Before reduction it behaves like a massive propagator, but its extra relations project the four endpoint states onto a single shared two-state:

2.2 Massless full edges: from four states to a shared two-state

Before the extra relations are used, a theta-carrying massless full edge behaves like a massive one: each endpoint carries a two-state h -factor, i.e. four states $\mathbf{F}^{(4)} = (F_{00}, F_{01}, F_{10}, F_{11})^T$. By (7), in the dimensionless paper basis

$$F_{01} = -F_{10}, \quad F_{11} = F_{00}, \quad (16)$$

where F_{ab} uses $h_1 = -k^{-1}\partial_\tau h_0$. The quotient to the shared two-state $\mathbf{g} = (F_{00}, F_{10})^T$ is the constant projection

$$\mathbf{F}^{(4)} = S_e \mathbf{g}, \quad S_e = \begin{pmatrix} 1 & 0 \\ 0 & -1 \\ 0 & 1 \\ 1 & 0 \end{pmatrix}, \quad P_e = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \end{pmatrix}, \quad P_e S_e = \mathbb{I}_2, \quad (17)$$

and any operator preserving the relation subspace reduces to $O_{\text{red}} = P_e O S_e$. In particular,

$$P_e(\sigma_2 \otimes \mathbb{I}_2)S_e = \sigma_2, \quad P_e(\mathbb{I}_2 \otimes \sigma_2)S_e = -\sigma_2, \quad (18)$$

so the two endpoints act on the *same* shared slot with opposite signs. This is the only regular reason why per-vertex factorization fails; the full-graph Kronecker-sum structure is preserved. The four-state contact vector of Eq. (3.65) of Chen and Feng [1] is $\mathbf{r}^{(4)} = (0, -1, 1, 0)^T = S_e |1\rangle_e$, so the contact selector in the shared basis is simply the fixed column $|1\rangle_e = (0, 1)^T$, with weights $w^{(u_e)} = +2i\sigma_e$, $w^{(v_e)} = -2i\sigma_e$ in the paper basis.

The package uses an algebraic quotient that separates the endpoint-derivative phases from the discrete states,

$$\mathbf{m}_{e,0} = \theta(\tau_{u_e} - \tau_{v_e})e^{-i\sigma_e k_e(\tau_{u_e} - \tau_{v_e})} + \theta(-(\tau_{u_e} - \tau_{v_e}))e^{+i\sigma_e k_e(\tau_{u_e} - \tau_{v_e})}, \quad (19)$$

$$\mathbf{m}_{e,1} = -\theta(\tau_{u_e} - \tau_{v_e})e^{-i\sigma_e k_e(\tau_{u_e} - \tau_{v_e})} + \theta(-(\tau_{u_e} - \tau_{v_e}))e^{+i\sigma_e k_e(\tau_{u_e} - \tau_{v_e})}. \quad (20)$$

Their relations are $F_{01}^J = -F_{10}^J$, $F_{11}^J = -F_{00}^J$. The two shared bases are related by the constant $D_{\sigma_e} = \text{diag}(1, i\sigma_e)$, under which the endpoint generators become $Q_{u_e}^J = +i\sigma_e k_e \sigma_1$ and $Q_{v_e}^J = -i\sigma_e k_e \sigma_1$, and the contact weights become the exact numbers $w_{(u_e)}^J = -2$, $w_{(v_e)}^J = +2$. The package diagonalizer for these σ_1 slots is the self-inverse Hadamard matrix

$$H = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}, \quad H\sigma_1 H^{-1} = \sigma_3, \quad (21)$$

so both bases produce the same energy letters (15). Every regular and contact matrix of the paper basis transforms to the package quotient by the constant similarity $O^J = B_s O^h B_s^{-1}$ (and $C_{st}^J = B_s C_{st}^h B_t^{-1}$ for contacts), with $B_s = \bigotimes D_{\sigma_e}$ on the massless slots.

3 Iterative reduction and the dlog-form differential equation

The remaining formulas are the two outputs of the package: the iterative reduction and the dlog-form differential equation.

3.1 Iterative reduction

The package's native integrals are

$$J_s(\mathbf{n}; \mathbf{a}) = \text{MSIntegral}[s, \mathbf{n}, \mathbf{a}], \quad \mathbf{n} \in \mathbb{Z}^{c_s}, \quad \mathbf{a} \in \{0, 1\}^{N_s}, \quad (22)$$

where c_s is the number of time components and N_s the number of two-dimensional slots of the sector. The sector s is identified by the fixed-length bit string of the root propagators (contracted = 0, uncontracted = 1; the top sector is all ones). The binary state order is $\mathbf{a}(m) = \text{IntegerDigits}(m, 2, N_s)$; deleting a contracted endpoint bit and inserting its complement at the other endpoint is done explicitly in the contact maps.

Writing the theta remaining term as a linear combination of lower-sector vectors $R_{v,s}(\mathbf{n}) = \sum_{e \ni v} C_{s,s/e}^{(v,e)} \mathbf{f}_{s/e}(\mathbf{n}_e)$, with $\mathbf{n}_e$ the child time shifts obtained by merging the shifts of the two endpoints, the one-step IBP at vertex v is

$$M_{1,v}(A_v + n_v) \mathbf{f}_s(\mathbf{n} - \mathbf{e}_v) + M_{0,v} \mathbf{f}_s(\mathbf{n}) + R_{v,s}(\mathbf{n}) = 0, \quad (23)$$

giving the lowering and raising steps

$$\mathbf{f}_s(\mathbf{n} - \mathbf{e}_v) = A_{-,v}(\mathbf{n}) \mathbf{f}_s(\mathbf{n}) - M_{1,v}^{-1} R_{v,s}(\mathbf{n}), \quad A_{-,v}(\mathbf{n}) = -M_{1,v}(A_v + n_v)^{-1} M_{0,v}, \quad (24)$$

$$\mathbf{f}_s(\mathbf{n} + \mathbf{e}_v) = A_{+,v}(\mathbf{n}) \mathbf{f}_s(\mathbf{n}) - U_s^{-1} \widetilde{M}_{0,v}^{-1} U_s R_{v,s}(\mathbf{n} + \mathbf{e}_v), \quad A_{+,v}(\mathbf{n}) = -U_s^{-1} \widetilde{M}_{0,v}^{-1} U_s M_{1,v}(A_v + n_v + 1). \quad (25)$$

Lowering has a pole exactly when the M_1 eigenvalue $\lambda_{1,v}(\mathbf{a}) = A_v + n_v - \sum_i (2\nu_i + 1) a_i$ vanishes, and raising has a pole exactly on the energy-letter surface $\kappa_{v;\mathbf{a},\mathbf{r}} = 0$; the zero matrix on the non-inverted direction never produces a pole. Pure massless vertices have $M_1 = \alpha_v \mathbb{I}$ with $\alpha_v = A_v + n_v$, and $\alpha_v = 0$ corresponds to the unregularized divergence of the defining $\Gamma(\alpha_v)$ integral; such cases are first regularized analytically ($\alpha_v \rightarrow \alpha_v + \delta$), reduced in the generic system, and expanded at $\delta \rightarrow 0$.

If a bundle of full lines shares one theta, one derivative produces one delta, and the product difference is a sum over odd nonempty subsets S of the bundle with coefficient $2^{1-|S|}$; each such subset is a simultaneous contact event. The base power of the merged component C is

$$A_C = \sum_{v \in C} A_v + \sum_{e \in E(C)} r_e + |C| - 1, \quad (26)$$

with r_e the raw contact power of the contracted edge. **MSReduce** iterates (24)–(25) along the contact DAG until all shifts vanish, and reports direction-sensitive singular layers.

The same building blocks also produce the differential equations. Differentiating the master vector with respect to the kinematic variables and reducing the result with the recurrence above yields the dlog-form potential in closed form:

3.2 The dlog-form differential equation

Following Chen et al. [2], the master integrals satisfy

$$dI = d\Omega \cdot I, \quad d\Omega = \sum_i \Omega_{k_i} dk_i, \quad (27)$$

with the diagonal matrices

$$\tilde{\Omega}_0 = \text{diag}[-i \log \kappa_{v;\mathbf{a},\mathbf{r}}], \quad \Omega_{\text{ex}} = \text{diag} \left[- \sum_i a_i (2\nu_i + 1) \log k_i \right], \quad (28)$$

and the fixed-sector potential

$$\Omega_{ss}^{\text{reg}} = \Omega_{\text{ex}} - i \sum_{v \in V_s} U_s^{-1} \tilde{\Omega}_0 U_s M_{1,v} (A_v + 1), \quad (29)$$

which is the full-graph version of Eq. (2.12) of Chen et al. [2]. Its letters are exactly the energy letters (15); no linear solution of IBP relations is needed.

The off-diagonal blocks use the remaining term $R^{(1)}$ of the raising relation,

$$M_{0,v} \mathbf{f}_s(\mathbf{e}_v) = M_{1,v} (A_v + 1) \mathbf{f}_s(0) - R_{v,s}(\mathbf{e}_v), \quad (30)$$

and generalize Eq. (3.68) of Chen and Feng [1] to

$$\Omega_{st}^{\text{contact}} = +i \sum_{v \in V_s} \chi_v(E_{st}) U_s^{-1} \tilde{\Omega}_0 U_s C_{st}^{(v,1)}, \quad t \prec s, \quad (31)$$

where $\chi_v(E_{st})$ is the sign of the vertex's pure energy block for pure-massive events, and $\chi_v(E_{st}) = (-1)^{N_0(E_{st})}$ when the event contains massless edges, N_0 being their number. For a contracted massless edge with shared bit position p_e , the intermediate-state sum collapses and the top-to-sub block is

$$(\Omega_{s,s/e})_{\mathbf{a},\hat{\mathbf{c}}} = 2i \left[(K_{u_e,s})_{\mathbf{a},\mathbf{c}'} - (K_{v_e,s})_{\mathbf{a},\mathbf{c}'} \right], \quad K_{v,s} = U_s^{-1} \tilde{\Omega}_0 U_s, \quad (32)$$

up to a prefactor collecting the absorbed sector normalization and pinch factors, where $\mathbf{c}'$ is the child state with the shared bit set to 1. If $R^{(1)}$ lands on a nonzero child shift, the program reduces each child state by the same recurrence and only writes the column into the connection when all columns close; otherwise it returns `contactShiftReductionFailed`. Ordering sectors by the number of remaining full edges, the complete connection is block triangular, with contacts pointing only from a parent to a strictly lower sector.

The physical endpoint derivatives satisfy $\mathcal{F}_{01} = -\mathcal{F}_{10}$, $\mathcal{F}_{11} = k^2 \mathcal{F}_{00}$; if the final basis uses them instead of the dimensionless h -states, the potential must be transformed by the gauge

$$\mathcal{A}_{\text{phys}} = dN N^{-1} + N \mathcal{A}_h N^{-1}, \quad N = \text{diag}(1, -k), \quad (33)$$

which contains $d \log k$ on the second entry.

It remains to fix the conventions that connect these formulas to the package interface.

3.3 Package conventions

The package's public integrals are normalized objects $J_s = \mathcal{N}_s I_s$ with time powers $(-\tau)^A$. In the normalized basis the sector diagonal block acquires $d \log \mathcal{N}_s \mathbb{I}_s$ and the contact blocks are multiplied by $\mathcal{N}_s / \mathcal{N}_t$. Only when this ratio is a kinematics-independent constant is (31) directly a constant-residue dlog. `NuConvention` is fixed at initialization. The `MSInitVertexFamily` input uses the compact `ki/nui` form:

```
ctx = MSInitVertexFamily[<|
  "ki" -> {k0,k1,...}, "nui" -> {A0,nu1,...},
  "hankelBranches" -> {1,2,...}
|>];
```

where `k0,A0` are the vertex phase energy and base time power, and the remaining entries give the h -block momenta and orders.

4 Boundary conditions by nested blow-up

The last ingredient for numerical evaluation is the boundary data at the large-damping-energy limit. With $\tau_v = -t_v < 0$ and the damping energy $K_v = ip_v > 0$, the vertex exponent is $e^{ip_v \tau_v} = e^{-K_v t_v}$, so $K_v \rightarrow \infty$ localizes the integral to $\tau_v \rightarrow 0^-$. Choosing a strict total order of the damping energies,

$$K_{\sigma(1)} \gg K_{\sigma(2)} \gg \cdots \gg K_{\sigma(V)} \gg 1, \quad (34)$$

defines the nested coordinates

$$x_j = \frac{K_{\sigma(j+1)}}{K_{\sigma(j)}} \quad (1 \leq j < V), \quad x_V = \frac{1}{K_{\sigma(V)}}, \quad (35)$$

under which every theta becomes 0 or 1 ($\theta(\tau_u - \tau_v) \rightarrow 1$ iff $K_u \gg K_v$). Every K -dependent denominator of the connection has the affine form $D_{\alpha,s} = Q_{\alpha,s}(k) + \sum_v c_{\alpha,s;v} K_v$ with kinematics-independent coefficients; substituting $1/K_{\sigma(j)} = \prod_{r \geq j} x_r$ factors it into a coordinate monomial times a nonzero unit, which removes the degenerate $K_i \rightarrow \infty$ singularities simultaneously. The program verifies this explicitly: `MSBoundaryChartCertificate` aggregates all letters and normalization divisors, extracts the coordinate monomial powers, requires the strict transforms to be nonzero at $x = 0$, and checks the normal-crossing Jacobian condition. `RankOrder -> All` enumerates every strict chart, and `MSBoundaryData` refuses to continue unless the chosen chart is certified.

The boundary data at $K_v \rightarrow \infty$ are the Frobenius leading branches $\{a, b, C\}$ of the master integrals, obtained per sector from the indicial system

$$(\lambda_{s,\mathbf{a}} \mathbb{I} - R_0) v_{s,\mathbf{a}} = 0 \quad (36)$$

of the pulled-back connection, with physical weights built from the component base powers, normalizations and endpoint coefficients (the two-vertex G^{++} example of Chen et al. [2], Sec. 4, is reproduced as a special case). `FlintNDE` then transports each exact branch from the regular-singular point to a finite match point, sums the branches with their weights, and transports along the affine pullback of the dlog connection to the user's ordinary point. Save points declared by `FlintNDESavePoints -> {{coord,"save"},...}` are written immediately; regular-singular starts store reusable $\{a, b, C\}$ data. Uncertified boundary charts, unclosed shift reductions and backend capabilities beyond exact regular-singular transport all fail closed.

Part II

Using the package

5 Overview, requirements and loading

`MadStree` is a topology-driven formula producer that implements the formulas of Part I. It accepts

- an ordered dS tree topology with massive and/or massless full lines,
- a pure time-only incidence graph (cycles allowed, but no loop-momentum integration), or
- a single-vertex function family without a fake graph,

and produces, without first generating a large system of integration-by-parts (IBP) identities,

- the contact-reachable sectors and their ordered master integrals $J_s(\mathbf{n}; \mathbf{a})$ of (22),
- single-step and complete iterative reduction data (Sec. 3),

- the block-triangular dlog-form potential Ω ((29)–(31)),
- Frobenius boundary data at the $K_v \rightarrow \infty$ limit (Sec. 4),
- numerical master-integral values at user-specified ordinary points, through FlintNDE.

MadStree does not run loop-momentum integrations, Kira, or external reduction; anything that cannot be certified (boundary charts, shift reductions, backend singular-point capabilities) fails closed instead of degrading silently.

Before using the package we collect the system requirements and the loading commands.

5.1 Requirements and loading

5.2 System requirements

- A Wolfram Language kernel: Mathematica or the free Wolfram Engine (the package is a .wl package; no external Wolfram libraries are needed).
- Python 3.10 or newer with `python-flint` ≥ 0.6 installed, for the numerical backend. MadStree invokes Python through the `PythonExecutable` option (default command "python").

5.3 Loading the package

The version root is `.../package-MadStree/versions/MadStree-v0.5`. Add it to `$Path` and load with `Needs`:

```
madStreeVersionDir = ".../package-MadStree/versions/MadStree-v0.5";
AppendTo[$Path, madStreeVersionDir];
Needs["MadStree"];

```

Equivalently, load the kernel file directly,

```
$CharacterEncoding = "UTF-8";
Get[".../package-MadStree/versions/MadStree-v0.5/Kernel/MadStree.wl"];

```

or use the official entry `Get[".../package-MadStree/load_current.wl"]`, which loads the version recorded in `VERSION.INDEX.md` (currently v0.5). After loading, `MSFlintNDEConfiguration[]` returns an association whose `availableQ` must be `True`; it confirms that the embedded FlintNDE backend under `Vendor/FlintNDE` has been found.

6 Input and core computation

6.1 Tree topology: MSInitTree

A tree is given by an association of vertices and lines,

```
spec = <|
  "name" -> "masslessFullEdge",
  "vertices" -> {
    <|"id" -> v1, "energy" -> k1, "timePower" -> a1|>,
    <|"id" -> v2, "energy" -> k2, "timePower" -> a2|>
  },
  "lines" -> {
    <|"id" -> e1, "type" -> "masslessFull", "endpoints" -> {v1, v2},
    "momentum" -> q, "skType" -> "++", "nu" -> 1/2|>
  }
|>;
context = MSInitTree[spec];

```


Vertex fields:

field	meaning	note
id	vertex symbol	required
energy	damping energy variable $k_{0,v}$ (e.g. k1)	required
timePower	time-power variable A_v (e.g. a1)	required
phaseSign	vertex exponent sign	optional; default +1

Line fields:

field	meaning	note
id	line symbol	required
type	"massiveFull", "masslessFull", "massiveExternal", "massiveCross", "masslessCross"	required
endpoints	ordered pair of vertex ids	required
momentum	momentum variable k_e (e.g. q)	required
skType	"++" or "--"	required for full lines
nu	Bessel/Hankel order $\nu = \nu $	required for massive/full lines
masslessRepresentation	"Quotient" (default) or "RedundantH"	masslessFull only

The initialization option `NuConvention -> "Positive"` (default) or `"Negative"` chooses the prefactor sign of $h(\nu, 0; z) = z^{\pm\nu} H_\nu(z)$ introduced in Sec. 1; it is fixed for the whole context and cannot be changed per call.

6.2 Time-only graphs: MSInitTimeGraph

For a pure time-only graph the vertex and line fields are the same, but the graph may contain cycles:

```
cycleContext = MSInitTimeGraph[<|
  "vertices" -> {<|"id" -> t1, "energy" -> k1, "timePower" -> a1|>, ...},
  "lines" -> {<|"id" -> l12, "type" -> "masslessFull", "endpoints" -> {t1, t2},
    "momentum" -> q12, "skType" -> "++", "nu" -> 1/2|>, ...}
|>];
```

6.3 Single-vertex families: MSInitVertexFamily

No fake topology is needed for a single vertex. The compact input follows the reference **ki/nui** convention,

```
vertexContext = MSInitVertexFamily[<|
  "ki" -> {k0, k1, k2},
  "nui" -> {a0, nu1, nu2},
  "hankelBranches" -> {1, 2}
|>, NuConvention -> "Positive"];
```

where `First[ki]` is the pure vertex phase energy and `First[nui]` the base power of $(-\tau)$; the remaining entries give, position by position, the h -block momenta and $\nu = |\nu|$. The explicit alternatives **k0/timePower/hBlocks/exponentialBlocks** are accepted.

Once the context is initialized, the following functions expose the formula products of Part I.

6.4 Core computation

All functions take the initialized `context`.

function	purpose
<code>MSectors[context]</code>	contact-reachable sectors in deterministic order (keys, contracted lines, master counts)
<code>MSMasterIntegrals[context]</code>	ordered master integrals <code>MSIntegral[sectorKey, timeShifts, stateBits]</code>
<code>MSFormulaMatrices[context, key]</code>	per-component M_1 , M_0 , U , U^{-1} , energy letters κ
<code>MSDLogDE[context]</code>	block-triangular dlog potential Ω , letters, certification status
<code>MSReduce[expr, context]</code>	iterative reduction to the master basis
<code>MSContactMaps[context, key]</code>	parent-to-subsector contact maps
<code>MSWriteFormulaArtifacts[context]</code>	write formula products beside the caller

`MSectors[context]` returns the sector list. Each sector key is the fixed-length bit string of Sec. 3: "0" contracted, "1" uncontracted, in root propagator order; the top sector is all ones. Keys are strings, not integers; leading zeros are part of the identity.

`MSMasterIntegrals[context]` returns the ordered master integrals `MSIntegral[sectorKey, timeShifts, stateBits]`, i.e. the $J_s(\mathbf{n}; \mathbf{a})$ of (22). Their order is the unique authority for all matrices, boundary vectors and numerical results.

`MSFormulaMatrices[context, key]` returns, per vertex component, the matrices M_1 , M_0 , the diagonalizer U of (13)–(14), its inverse, and the energy letters κ of (15).

`MSDLogDE[context]` returns `omegaPotential`, the matrix Ω of Sec. 3.2 whose derivative along a path is the DE matrix, together with the letters, `letterMatrices`, `dlogResidual`, `constantResidueQ` and the certification `dlogStatus`. Only "certifiedByFormulaChecks" is a closed dlog and may be consumed numerically.

`MSReduce[expr, context, MasterBasis -> Automatic]` reduces finite linear combinations of legal `MSIntegrals` by (24)–(25) along the contact DAG. Results include `result`, `masterBasis`, `coefficientVector`, `masterRules`, `nonMasterResidual`, `remainingShiftedIntegrals` and `singularLayers`; a full reduction requires a zero residual and an empty shifted list.

7 Boundary data and numerical evaluation

With an initialized context, numerical evaluation proceeds in two steps: boundary generation and transport.

7.1 Generating the boundary

```
targetRules = {k1 -> -9 I, k2 -> -3 I, q -> 1, a1 -> 1, a2 -> 1};

certificate = MSBoundaryChartCertificate[context, targetRules, RankOrder -> All];

boundary = MSBoundaryData[context, targetRules,
  BoundaryScale -> 4, WorkingPrecision -> 40];
```

`MSBoundaryChartCertificate` verifies the nested blow-up chart of Sec. 4 (normal crossing, theta fixing, quotient checks) for the chosen strict energy rank; `RankOrder` $\rightarrow$ All checks every strict chart. `MSBoundaryData` consumes the certificate and returns the Frobenius leading branches $\{a, b, C\}$ at the $K_v \rightarrow \infty$ boundary. If the chart is not certified, it returns `BoundaryChartNotCertified` and does not proceed to numerics.

7.2 Numerical transport

```
targetValue = MSEvaluateTree[context, targetRules,
  BoundaryScale -> 4,
  WorkingPrecision -> 40,
  TransportOrder -> 80,
  ReferenceTransportOrder -> 104,
  TargetRelativeError -> "1e-20"];
```

```
targetValue["values"]
```

For a single-vertex family use `MSEvaluateVertexFamily[vertexContext, targetRules]`. Principal options:

option	meaning
<code>BoundaryScale</code>	distance of the finite match point from the boundary (> 1)
<code>BoundarySeriesOrder</code>	truncation order of the boundary series
<code>RankOrder</code>	vertex order by damping energy K_v (default: from the target point)
<code>WorkingPrecision</code>	working precision digits for the backend
<code>TransportOrder</code>	/ local-series orders of the primary/reference chains
<code>ReferenceTransportOrder</code>	
<code>TargetRelativeError</code>	target relative error between the two chains
<code>PythonExecutable</code>	Python command for the FlintNDE adapter (default "python")
<code>FlintNDESavePoints</code>	unnamed save points $\{\{\text{coord}, \text{"save"}\}, \dots\}$
<code>MSRuntimeDirectory</code>	caller directory for temporary files (default: script directory)

7.3 Reading the result

`targetValue` is an association with `status` (must be "computed"), `values` (the master vector at the target, in master order), `masters`, `masterDigest`, `boundary`, `pathRules`, and `flintNDE`. For the generic singular route, `flintNDE` has schema `"madstree_flintnde_singular_then_ordinary_v1"` with `singularLaunch` and `ordinaryTransport`; the primary/reference relative difference is then inside `ordinaryTransport`. A route-robust access is

```
relativeDifferenceInf = Lookup[
  targetValue["flintNDE"], "relativeDifferenceInf",
  Lookup[targetValue["flintNDE"], "ordinaryTransport"],
  "relativeDifferenceInf", Missing["NotAvailable"]];
targetValue["flintNDE", "targetRelativeErrorMet"]
```

8 Outputs, examples, validation and common pitfalls

8.1 Output organization

- Transient JSON and Python caches: `results_temp/flintnde_transport/` beside the calling script; deleted after success, kept on failure for diagnostics.
- Save points: `results/flintnde_save_points/run-UUID/`; each `{coordinate,"save"}` point is written immediately (`flintnde_save_001.json, ...`), and the aggregate `madstree_flintnde_save_points.json` only after full success.
- Formula artifacts: `MSWriteFormulaArtifacts[context]` writes `masters.wl`, `recurrence_metadata.wl`, `dlog_de.wl` and `manifest.wl` to `results/madstree_formula/run-UUID/`.

The package source directory never receives run artifacts.

The distribution ships three examples that exercise the complete workflow.

8.2 Examples

The v0.5 distribution ships three runnable examples under `versions/MadStree-v0.5/Examples/`:

- `01_massless_full_edge.wl`: the massless quotient of Sec. 2.2, masters, recurrence, dlog and the automatic boundary/numerical entry;
- `02_vertex_family_reduction.wl`: single-vertex family input, local tensor inverses, and finite-combination reduction;
- `03_time_only_cycle_chart.wl`: time-only cycle input, common-theta contacts, and all strict-rank chart certificates.

Run them in a Mathematica front end section by section, or with

```
wolframscript -file ../Examples/01_massless_full_edge.wl
```

The following validation status and fail-closed guarantees define the supported domain of this release.

8.3 Validation status and limitations

- v0.5 passed all eleven development tests and the three examples (exit code 0).
- Independent validations T1–T6 passed with counts 24/24, 12/12, 18/18, 15/15, 17/17, 16/16; T3 (the two-vertex G^{++} system of Sec. 4) reproduces the independent 2411 route with exactly zero connection residuals.
- The embedded FlintNDE test suite passes 76/76.

Limitations and fail-closed guarantees:

- No loop-momentum integration, no sampled IBP, no Kira or external reduction.
- `dlogStatus` must be `"certifiedByFormulaChecks"`; otherwise the DE is not consumed.
- Uncertified boundary charts, unclosed shift reductions, and singular points beyond the back-end's certified capabilities all fail closed; ordinary Taylor transport is never used across singular points.
- The production boundary never uses defining integrals; finite-point defining integrals exist only inside independent validation programs.

Finally, we collect the most common mistakes.

8.4 Common pitfalls

1. **Convention:** `NuConvention` is chosen at initialization only ("Positive" by default). To compare with the formulas of Chen et al. [2], initialize with `NuConvention -> "Negative"`, which selects the minus sign in (2).
2. **Sector keys:** fixed-length strings; leading zeros and the string type are part of the identity (Sec. 3).
3. **Save points:** only `{{coordinate,"save"},...}` is accepted; no names, and coordinates must be exact Gaussian rationals.
4. **Backend environment:** the Python invoked by `PythonExecutable` must have `python-flint` installed and must be able to import the embedded `flintnde`; otherwise transport returns `FlintNDEProcessFailed` (inspect the failure's `backend` field or the retained `results_temp` JSON).
5. **Result access:** for the singular route, `relativeDifferenceInf` lives inside `ordinaryTransport`; use the fallback lookup of Sec. 7.
6. **Encoding:** if loading fails with syntax errors, set `$CharacterEncoding = "UTF-8"` before loading.

We close with a brief summary of the package's data flow. `MadStree` implements the formulas above as a single data flow: the building-block registry directly produces the Kronecker-sum matrices (13)–(14), the massless quotient (17), the iterative recurrence (24)–(25), the block-triangular dlog potential (29)–(31), and the nested-blow-up boundary data, without generating or solving a large IBP system. The same registry feeds the numerical backend `FlintNDE`, which transports the masters from the $K_v \rightarrow \infty$ boundary to user-supplied ordinary points.

References

- [1] Jiaqi Chen and Bo Feng. Towards Systematic Evaluation of de Sitter Correlators via Generalized Integration-By-Parts Relations. *JHEP*, 06:199, 2024. doi: 10.1007/JHEP06(2024)199.
- [2] Jiaqi Chen, Bo Feng, and Yi-Xiao Tao. Multivariate hypergeometric solutions of cosmological (dS) correlators by dlog-form differential equations. *JHEP*, 03:075, 2025. doi: 10.1007/JHEP03(2025)075.